

Python Introduction and Installation



A programming language created by Guido van Rossum and was made available in 1991.

Python can be used for:

- Web programs,
- Scripts,
- Math,
- Big data management,
- Data analytics

Python Introduction and Installation

- Why Python?
- Portable
 - Windows, Mac, Linux, Raspberry Pi, etc.
- “common sense” syntax
 - Easy to learn
 - Simplicity
- Interpretive programming language
- Highly forgiven
 - procedural, functional, object-orientated

Python Installation

- Anaconda
 - Python distribution and related libraries

Python Installation Go to [Anaconda.com/download](https://anaconda.com/download)

← → ↻ anaconda.com/products/individual ☆ ⚙ (



Individual Edition

Your data science toolkit

With over 25 million users worldwide, the open-source Individual Edition (Distribution) is the easiest way to perform Python/R data science and machine learning on a single machine. Developed for solo practitioners, it is the toolkit that equips you to work with thousands of open-source packages and libraries.

Anaconda Individual Edition

Download 

For Windows

Python 3.8 • 64-Bit Graphical Installer • 477 MB

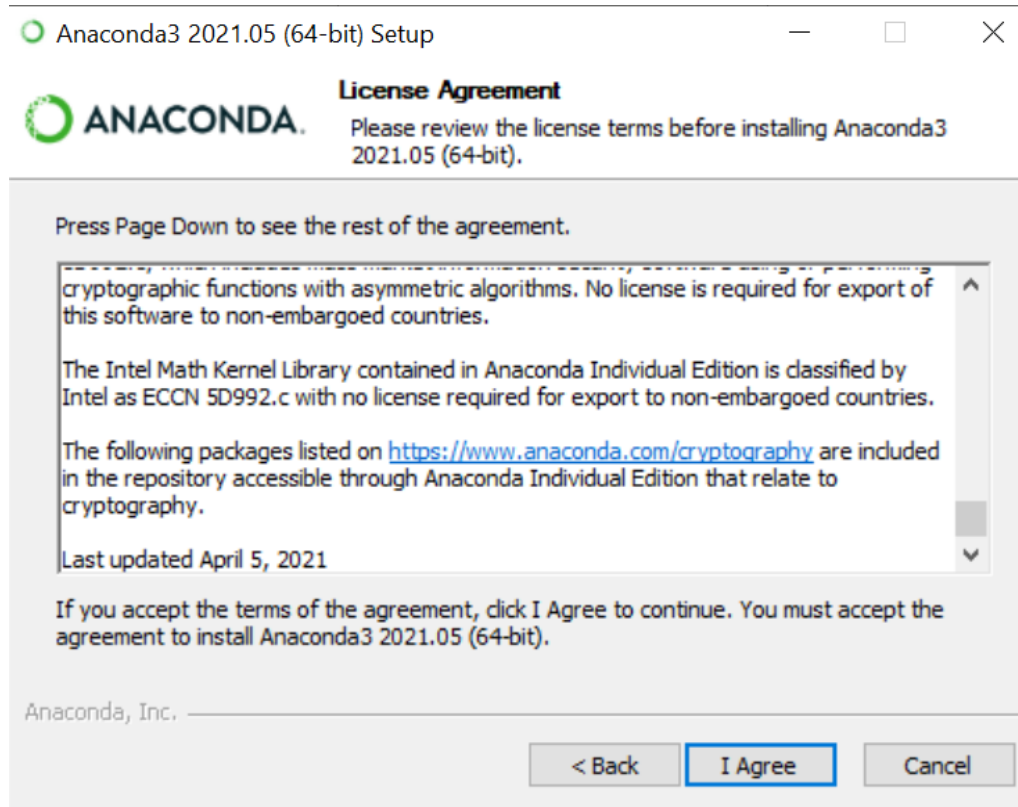
Get Additional Installers



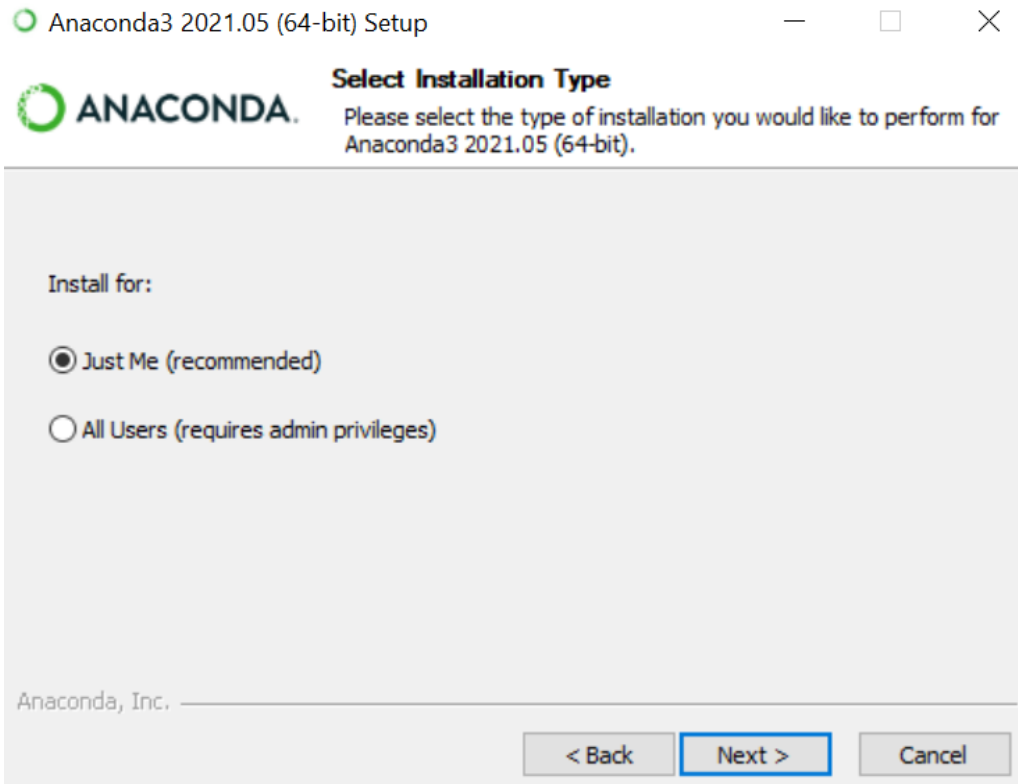
Python Installation Go to [Anaconda.com/download](https://anaconda.com/download)



Python Installation



Python Installation



Python Installation

Anaconda3 2021.05 (64-bit) Setup



Choose Install Location

Choose the folder in which to install Anaconda3 2021.05 (64-bit).

Setup will install Anaconda3 2021.05 (64-bit) in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.

Destination Folder

G:\Thinkpad\bhuang\Anaconda\

Browse...

Space required: 2.9GB

Space available: 587.5GB

Anaconda, Inc.

< Back

Next >

Cancel

Python Installation

Anaconda3 2021.05 (64-bit) Setup — □ ×



Advanced Installation Options

Customize how Anaconda integrates with Windows

Advanced Options

☐ Add Anaconda3 to my PATH environment variable

Not recommended. Instead, open Anaconda3 with the Windows Start menu and select "Anaconda (64-bit)". This "add to PATH" option makes Anaconda get found before previously installed software, but may cause problems requiring you to uninstall and reinstall Anaconda.

☒ Register Anaconda3 as my default Python 3.8

This will allow other programs, such as Python Tools for Visual Studio, PyCharm, Wing IDE, PyDev, and MSI binary packages, to automatically detect Anaconda as the primary Python 3.8 on the system.

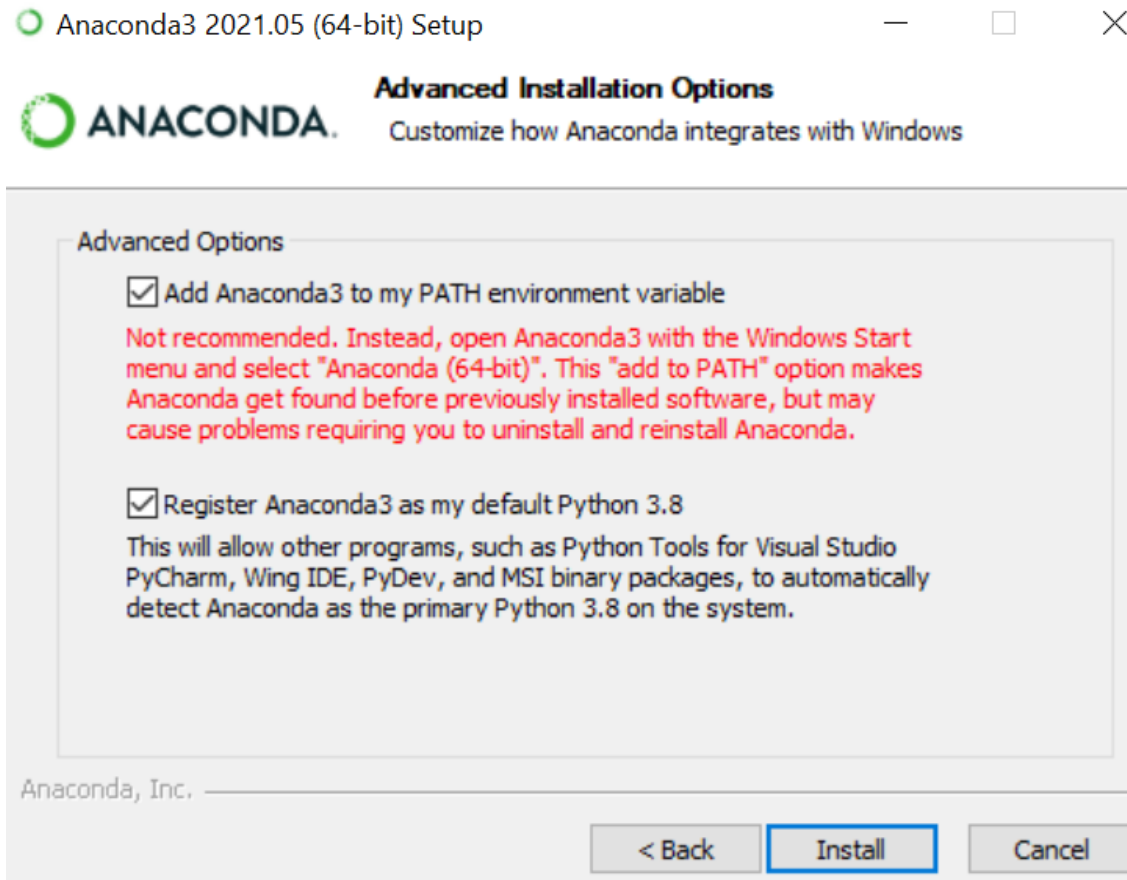
Anaconda, Inc.

< Back

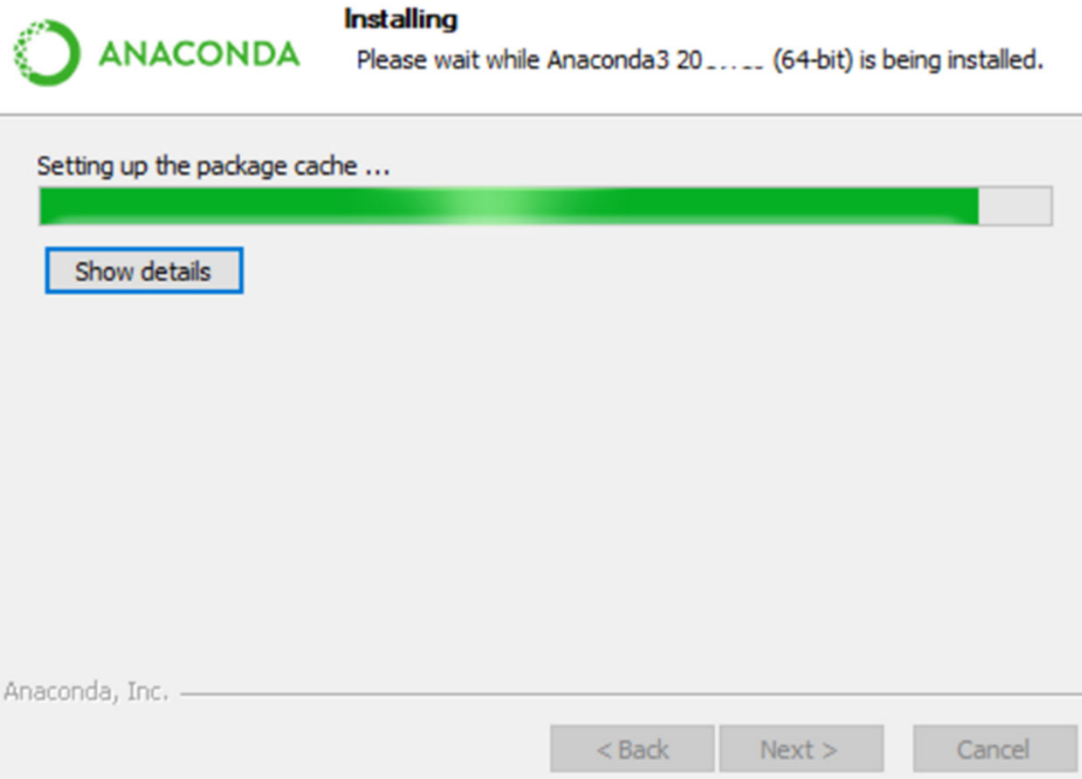
Install

Cancel

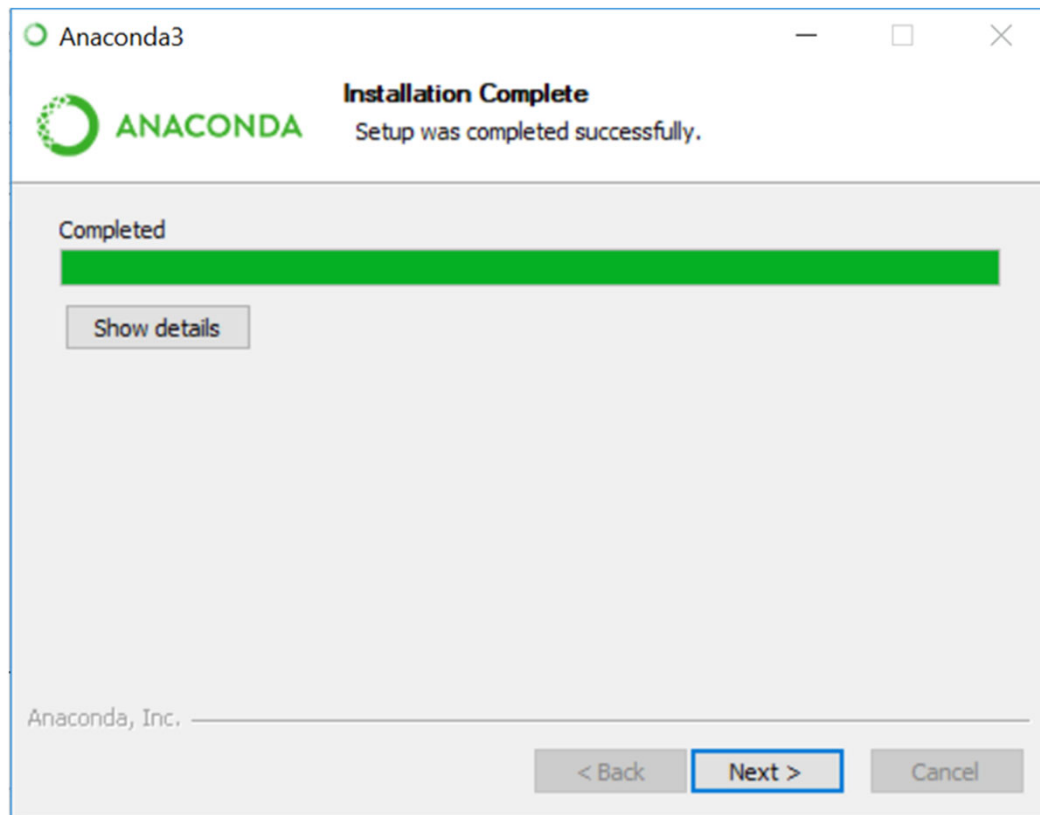
Python Installation



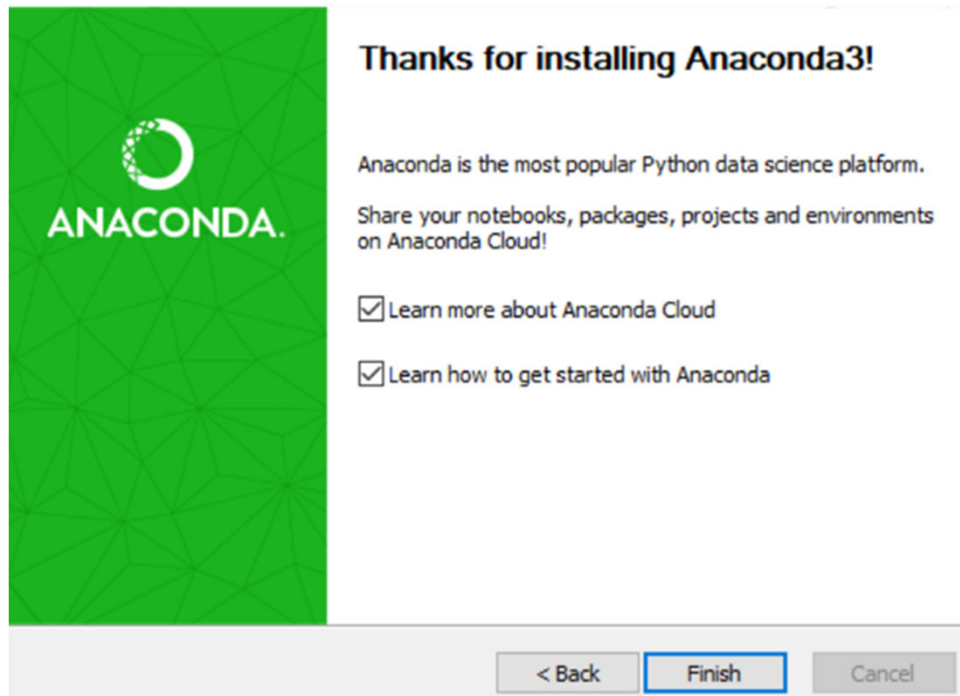
Python Installation



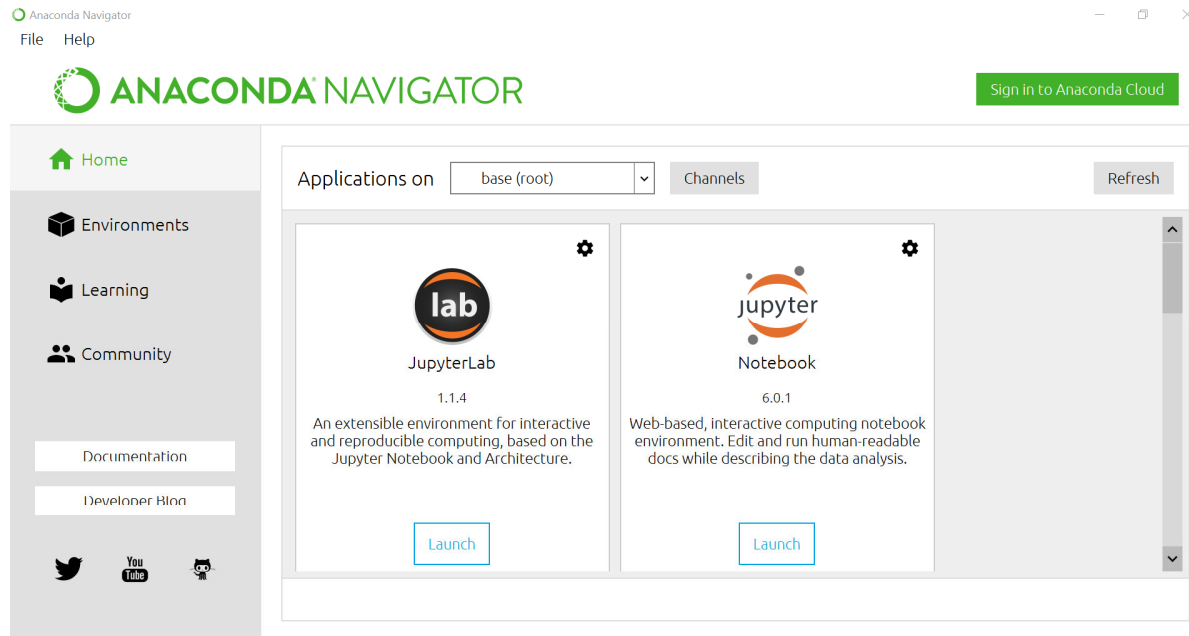
Data Science: Python Installation



Python Installation



Python Installation



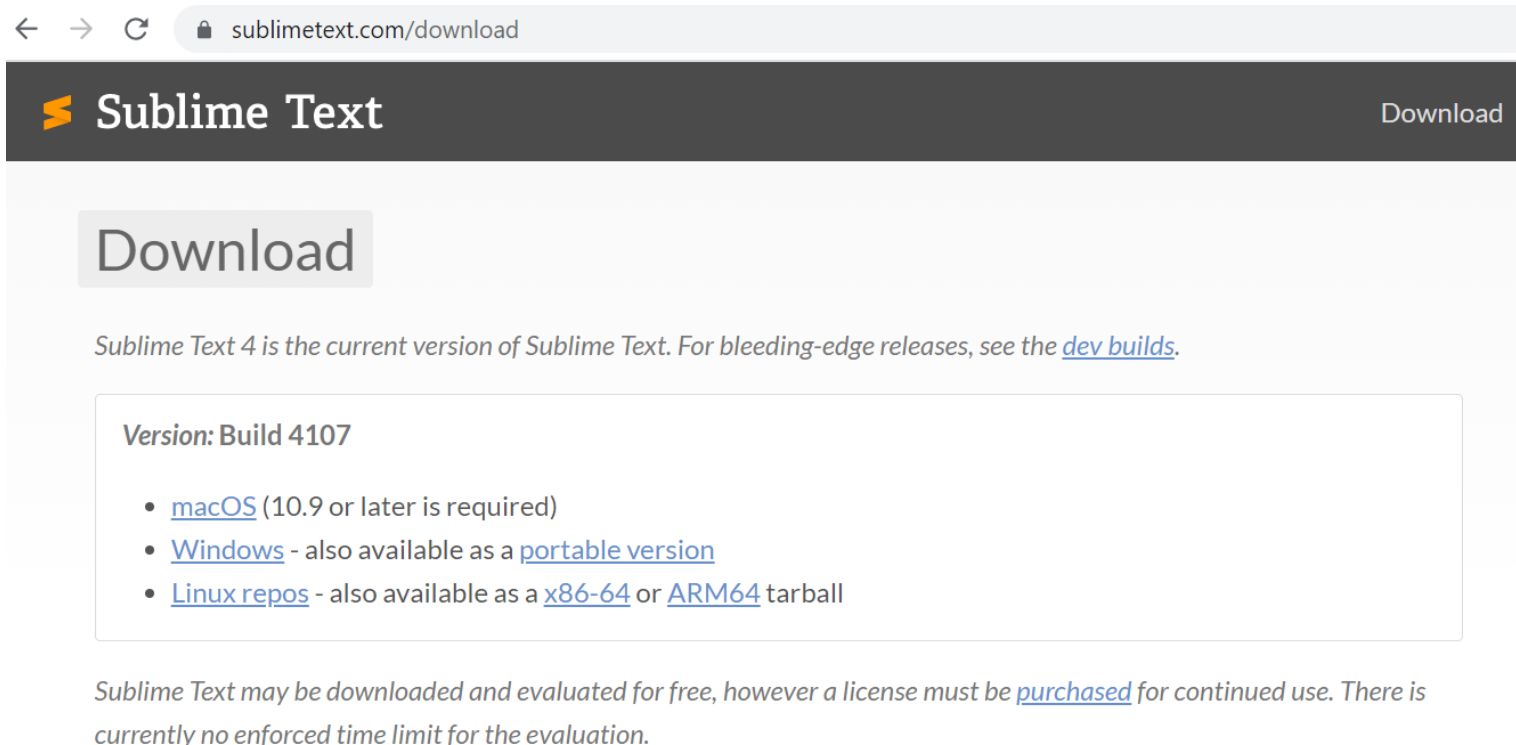
Python Introduction and Installation



Python Sublime Text Editor



Python Sublime Text Editor • Go to <https://www.sublimetext.com/download>



The screenshot shows the Sublime Text download page. At the top, there is a dark header with the Sublime Text logo and a 'Download' button. Below the header, the word 'Download' is prominently displayed in a large, bold font. A paragraph of text states that Sublime Text 4 is the current version and provides a link to 'dev builds'. A box contains the version information 'Version: Build 4107' and a list of operating systems with links to their respective download pages. At the bottom, a paragraph explains that the software can be downloaded and evaluated for free, but a license must be purchased for continued use.

← → ↻ [sublimetext.com/download](https://www.sublimetext.com/download)

 **Sublime Text** [Download](#)

Download

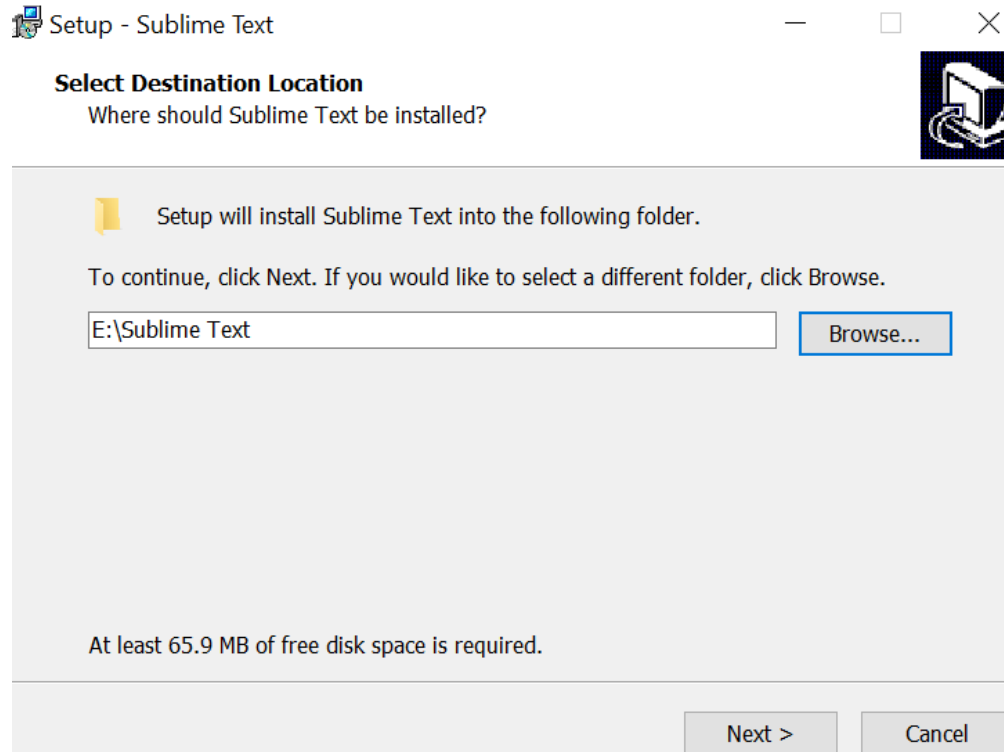
Sublime Text 4 is the current version of Sublime Text. For bleeding-edge releases, see the [dev builds](#).

Version: Build 4107

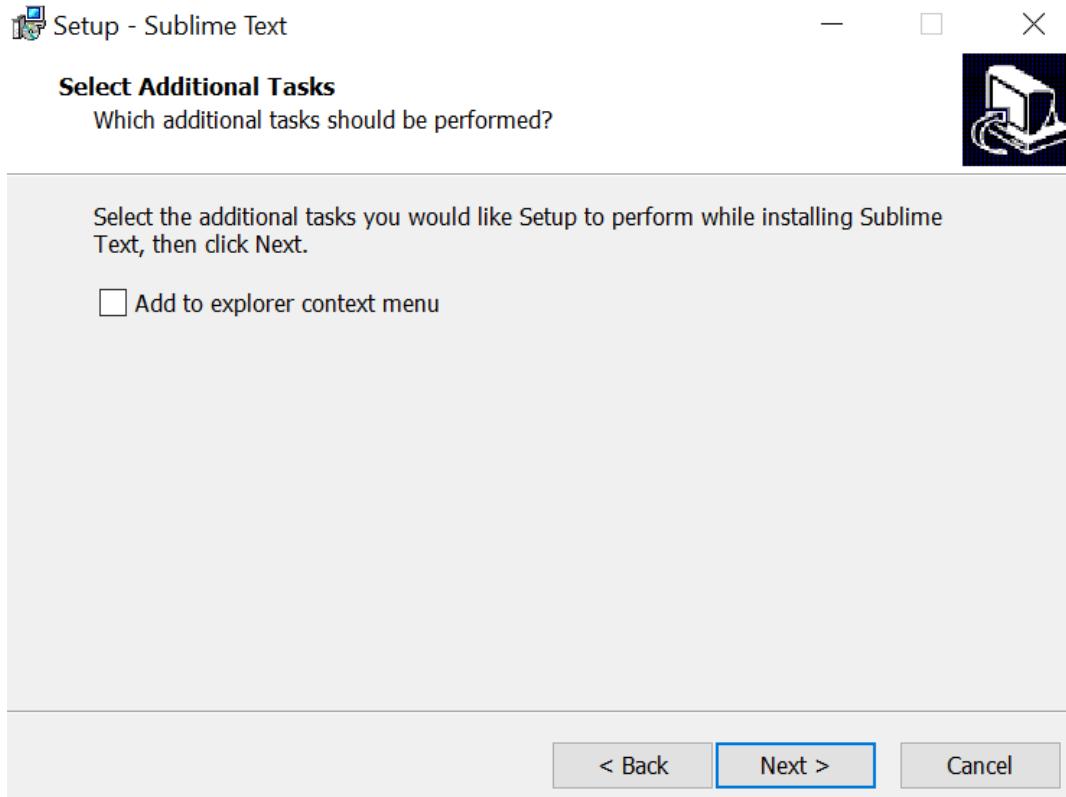
- [macOS](#) (10.9 or later is required)
- [Windows](#) - also available as a [portable version](#)
- [Linux repos](#) - also available as a [x86-64](#) or [ARM64](#) tarball

Sublime Text may be downloaded and evaluated for free, however a license must be [purchased](#) for continued use. There is currently no enforced time limit for the evaluation.

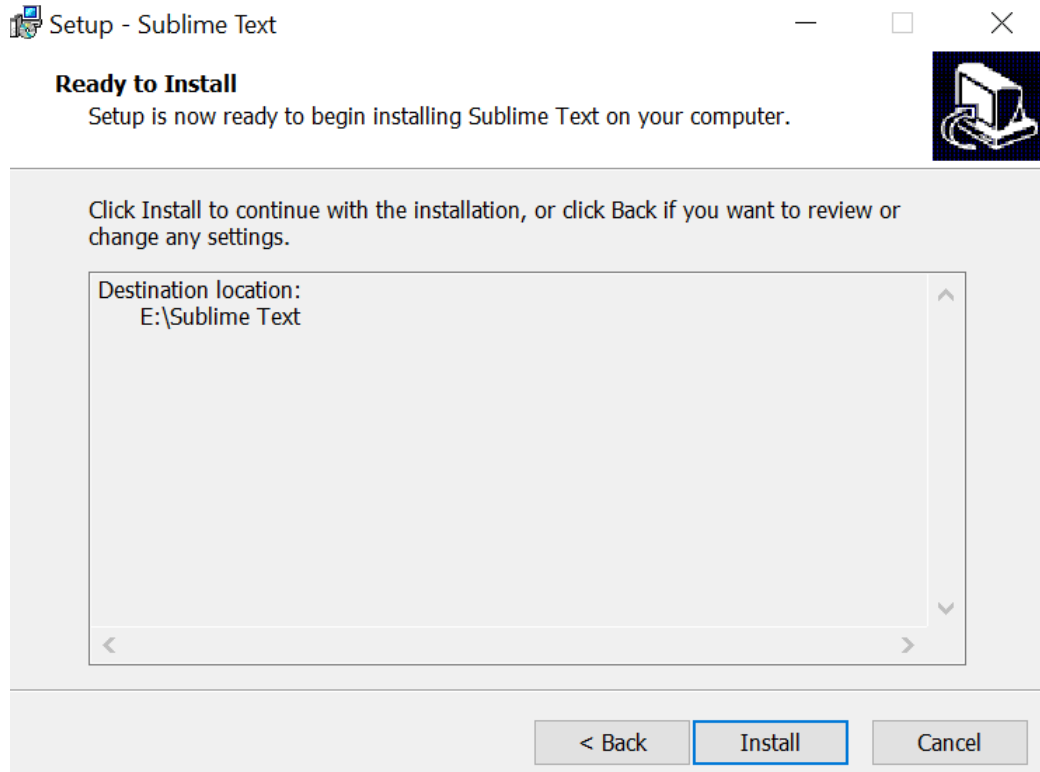
Python Sublime Text Editor



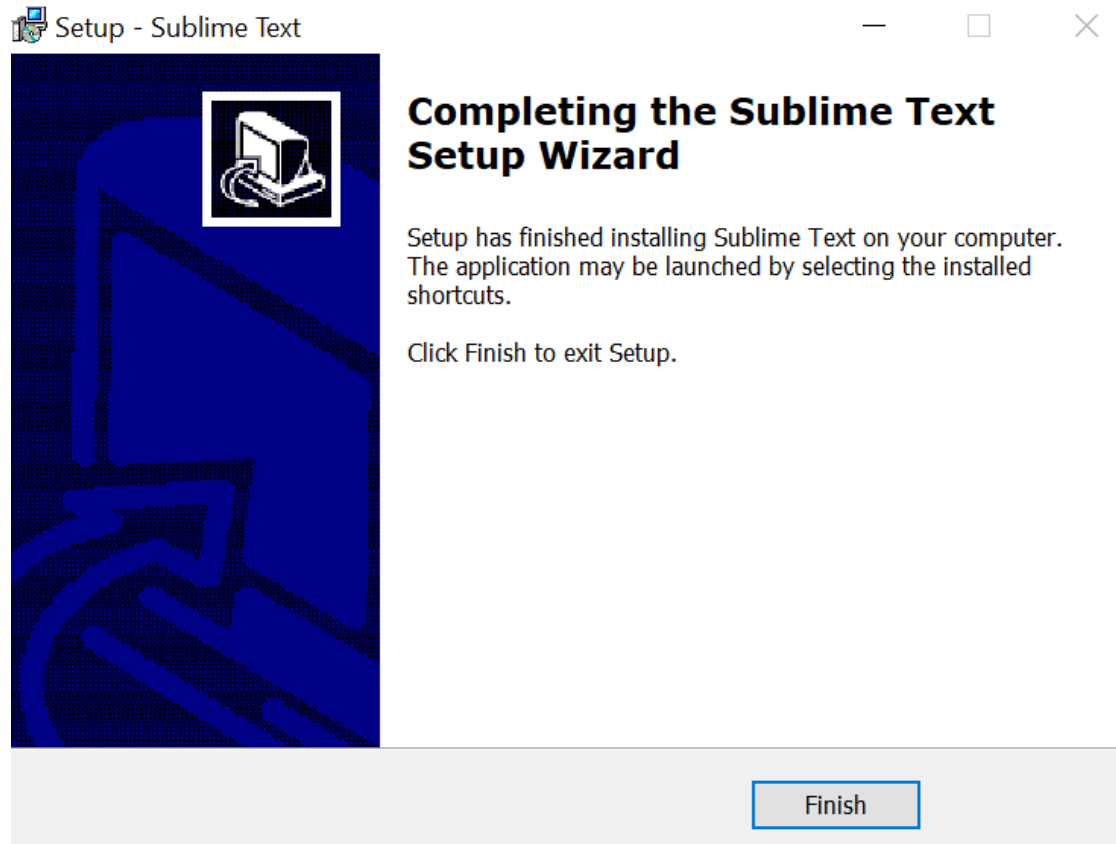
Python Sublime Text Editor



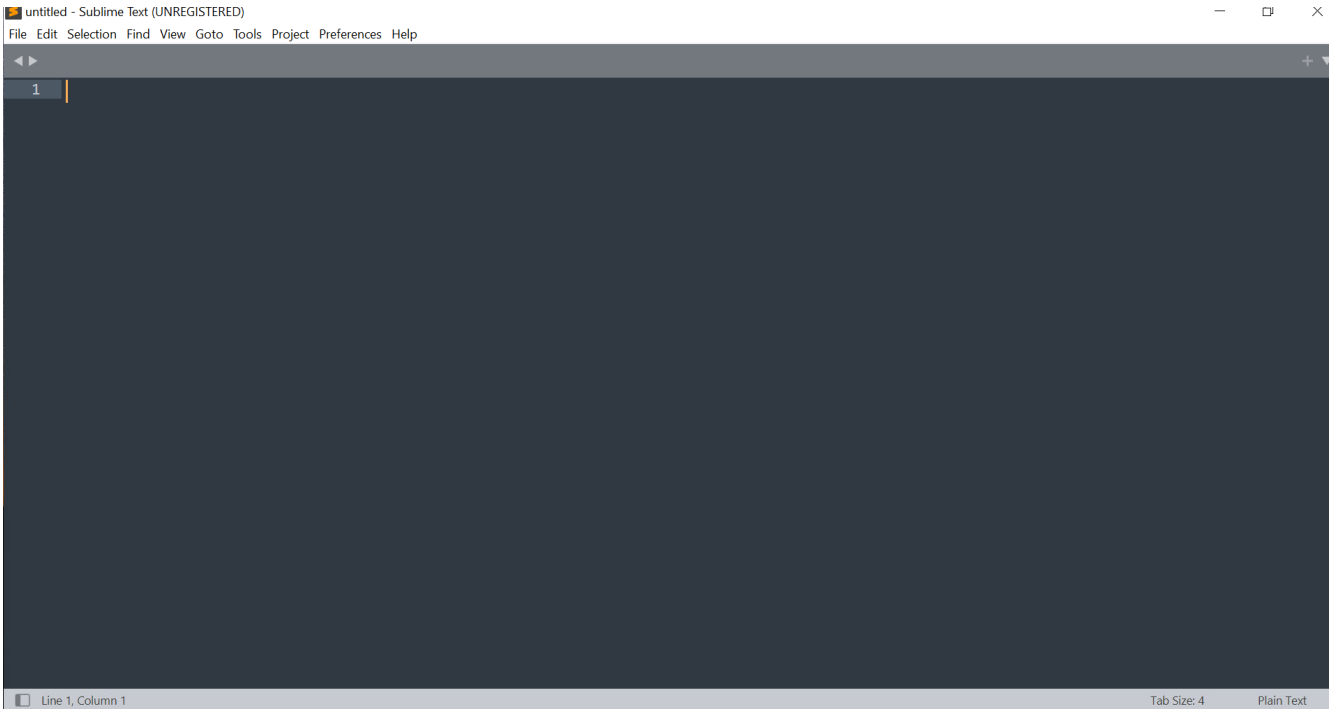
Python Sublime Text Editor



Python Sublime Text Editor



Python Sublime Text Editor



Data Science: Python Sublime Text Editor



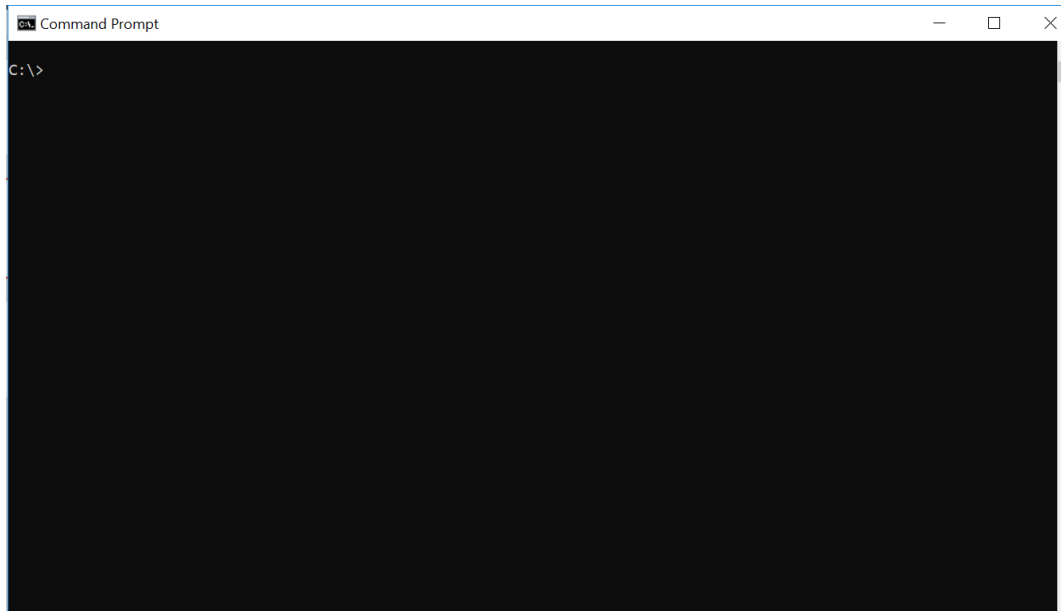
Data Science: Python Prompt via Command Prompt



Python Prompt via Command Prompt

To get to the Python Prompt:

1. Launch Command Prompt (The Mac OS equivalent is Terminal)
 1. Go to bottom left corner of Windows where you can type
 1. Enter CMD then select Command Prompt from the list



Python Prompt via Command Prompt

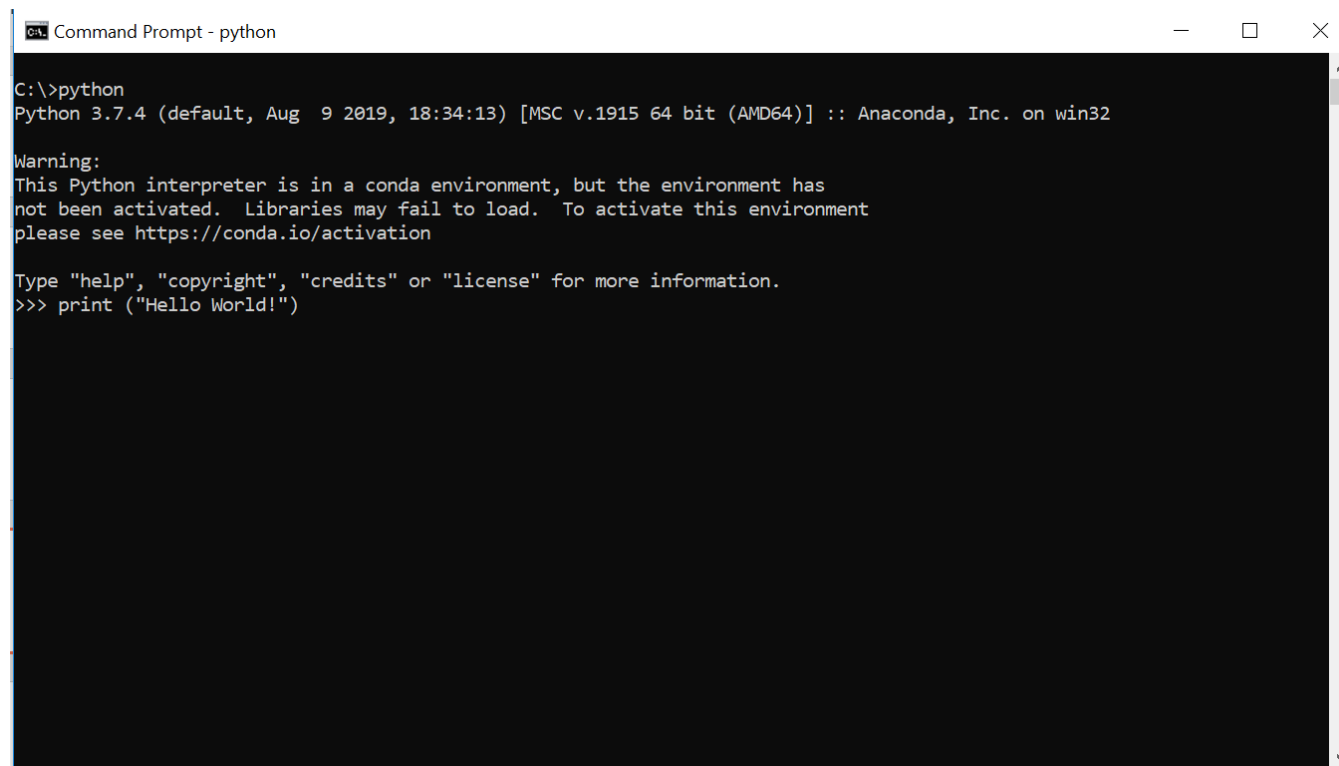
To get to the Python Prompt from Command Prompt:

1. Type python at the prompt then hit ENTER
2. The >>> prompt is the Python Prompt. From here you can write and run Python code

Python Prompt via Command Prompt

Let's write our first Python program via Python prompt

1. `print ("Hello World!")`



```
C:\>python
Python 3.7.4 (default, Aug  9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32

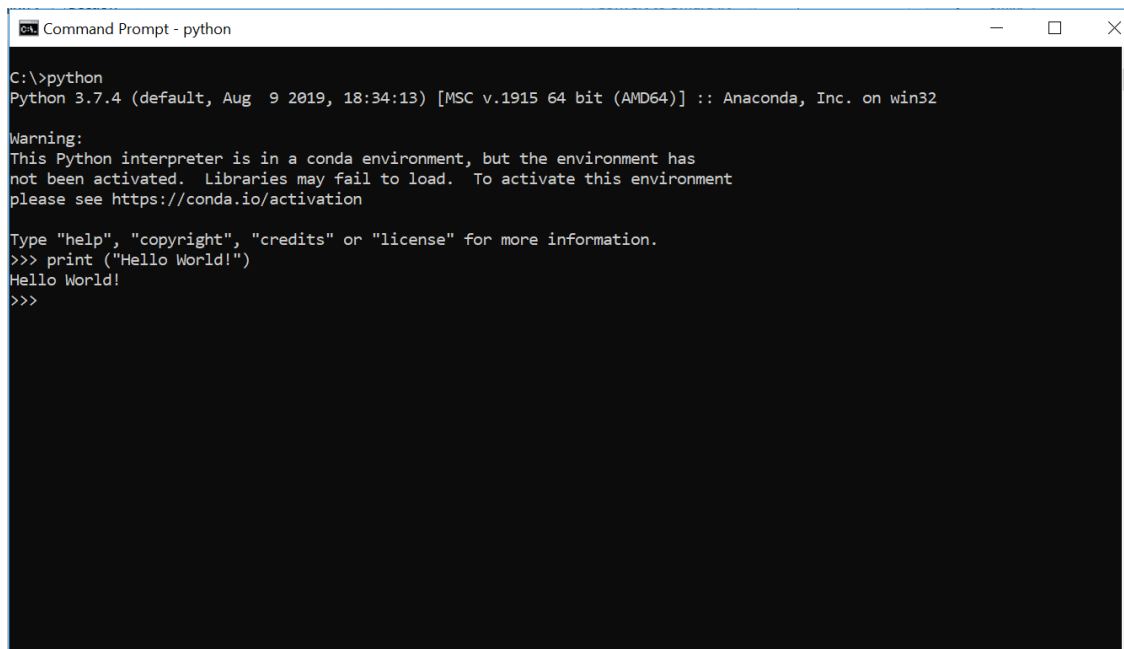
Warning:
This Python interpreter is in a conda environment, but the environment has
not been activated. Libraries may fail to load. To activate this environment
please see https://conda.io/activation

Type "help", "copyright", "credits" or "license" for more information.
>>> print ("Hello World!")
```

Python Prompt via Command Prompt

Let's write our first Python program via Python prompt

1. Python prompt displays (prints) your program output on the screen



```
Command Prompt - python
C:\>python
Python 3.7.4 (default, Aug  9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32

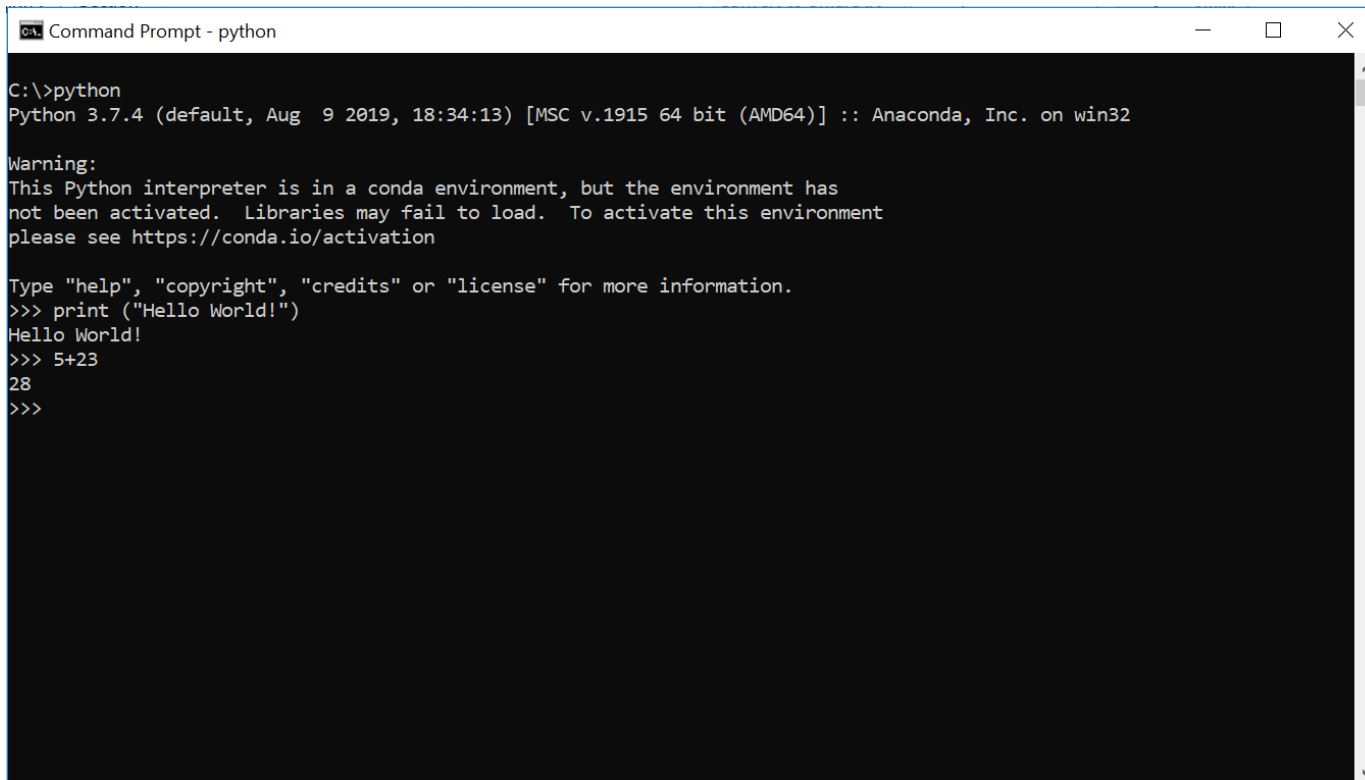
Warning:
This Python interpreter is in a conda environment, but the environment has
not been activated. Libraries may fail to load. To activate this environment
please see https://conda.io/activation

Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World!")
Hello World!
>>>
```

Python Prompt via Command Prompt

Let's use Python Prompt just like a calculator

1. Enter 5 + 23 at the Python Prompt
2. Python Prompt automatically displays the results of 5+23



```
C:\>python
Python 3.7.4 (default, Aug  9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32

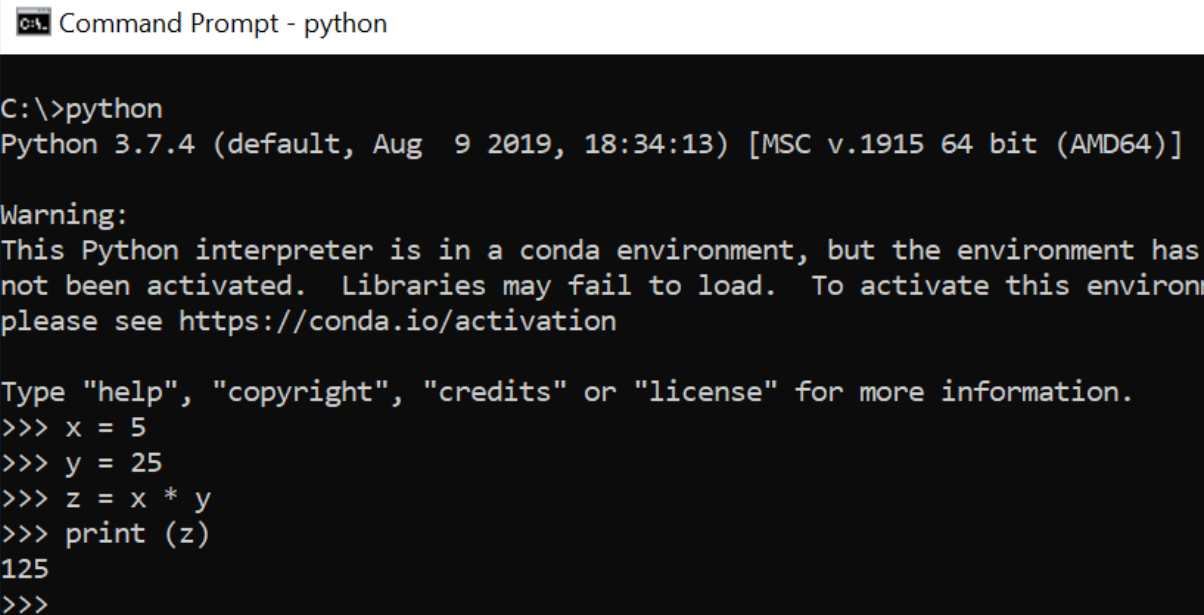
Warning:
This Python interpreter is in a conda environment, but the environment has
not been activated. Libraries may fail to load. To activate this environment
please see https://conda.io/activation

Type "help", "copyright", "credits" or "license" for more information.
>>> print ("Hello World!")
Hello World!
>>> 5+23
28
>>>
```

Python Prompt via Command Prompt

Let's write another program that will assign values to two variables then multiply these two variables to generate result and store it in a third variable

1. Enter `x = 5` at the Python Prompt
2. Enter `y = 25` at the Python Prompt
3. Enter `z = x * y` at the Python Prompt
4. Enter `print (z)` at the Python Prompt



```
C:\>python
Python 3.7.4 (default, Aug  9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)]

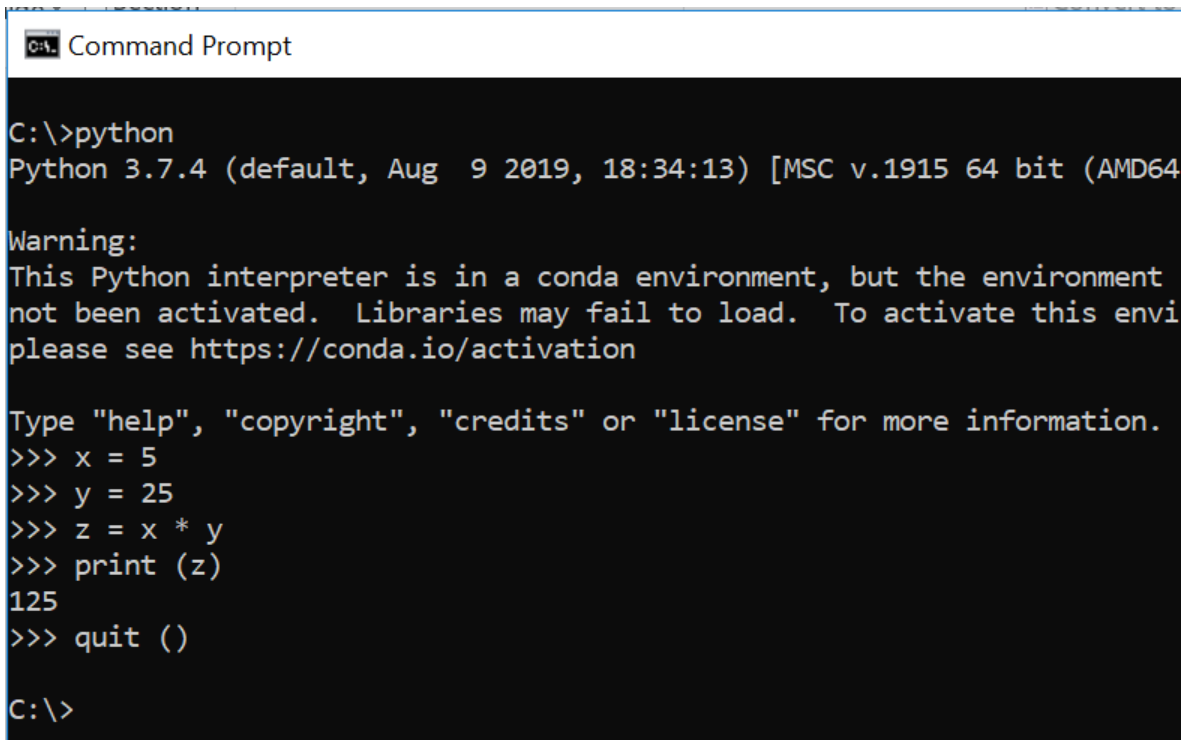
Warning:
This Python interpreter is in a conda environment, but the environment has
not been activated. Libraries may fail to load. To activate this environment
please see https://conda.io/activation

Type "help", "copyright", "credits" or "license" for more information.
>>> x = 5
>>> y = 25
>>> z = x * y
>>> print (z)
125
>>>
```

Python Prompt via Command Prompt

To end the Python Prompt session:

1. Enter quit()

A screenshot of a Windows Command Prompt window titled "Command Prompt". The window has a black background with white text. The prompt is "C:\>". The user has entered "python", which has started a Python 3.7.4 interpreter. The interpreter shows a warning message about the conda environment not being activated. The user has entered several lines of Python code: "x = 5", "y = 25", "z = x * y", and "print (z)". The output of the print statement is "125". Finally, the user has entered "quit ()", and the prompt has returned to "C:\>".

```
C:\>python
Python 3.7.4 (default, Aug  9 2019, 18:34:13) [MSC v.1915 64 bit (AMD64)]
Warning:
This Python interpreter is in a conda environment, but the environment
has not been activated. Libraries may fail to load. To activate this environment
please see https://conda.io/activation

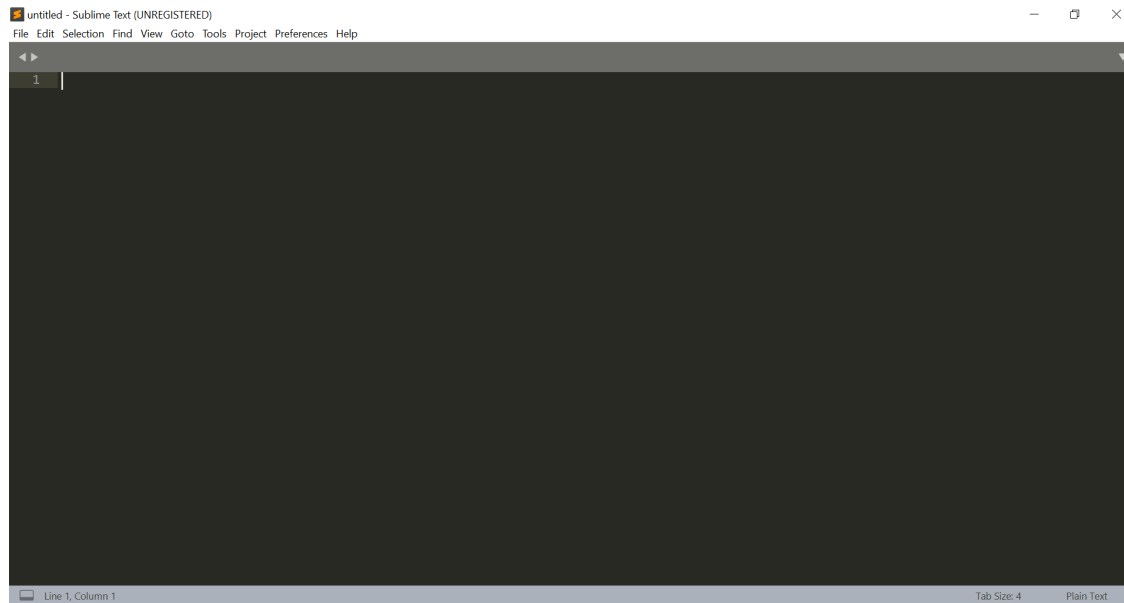
Type "help", "copyright", "credits" or "license()" for more information.
>>> x = 5
>>> y = 25
>>> z = x * y
>>> print (z)
125
>>> quit ()

C:\>
```

Python Programming Using Editor



Python Programming Using Editor



Python Programming Using Editor



Data Science: Python Programming Arithmetic Operators



Data Science: Python Programming Arithmetic Operators

- + addition
- - subtraction
- * multiplication
- / division
- % mod
- ** to the power of

My Dear Aunt Sally applies!

Python Programming Arithmetic Operators

 Command Prompt - python

```
Microsoft Windows [Version 10.0.17134.950]
(c) 2018 Microsoft Corporation. All rights reserved.

P:\>python
Python 3.7.4 (default, Aug  9 2019, 18:34:11)
Warning:
This Python interpreter is in a conda environment
that has not been activated. Libraries may fail to load
in some cases. For more information, please see
https://conda.io/activation

Type "help", "copyright", "credits" or "license()" for
>>> 1 + 1
2
>>> 1 * 2
2
>>> 1 / 2
0.5
>>> 1 - 1
0
>>> 1 % 2
1
>>> 5**2
25
>>> 5**3
125
>>>
```

[illegible]

Data Science: Python Data Types



Python: For Loop

For loop the traditional thinking....

For counter from 1 to 100

Do.....

For Counter = 0, add 1 to the counter for each iteration until counter is less than X

Python: For Loop

a for loop iterates over a sequence

a list, a tuple, a dictionary, a set, or a string

For example:

```
genre = ["Action", "Adventure", "Animation", "Biography", "Comedy", "Crime",  
"Drama", "Family", "Fantasy", "History", "Horror", "Music", "Musical", "Mystery",  
"Romance", "Sci-Fi", "Sport", "Thriller", "War", "Western"]
```

```
count = 0
```

```
for value in genre:
```

```
    count = count + 1
```

```
print("There are ", count, " genres of movies in the list")
```

Python: For Loop

a for loop iterates over a sequence

a list, a tuple, a dictionary, a set, or a string

For example:

```
genre = ["Action", "Adventure", "Animation", "Biography", "Comedy", "Crime",  
"Drama", "Family", "Fantasy", "History", "Horror", "Music", "Musical", "Mystery",  
"Romance", "Sci-Fi", "Sport", "Thriller", "War", "Western"]
```

```
for value in genre:  
    print(value)
```

Python: For Loop

a for loop iterates over a sequence

a list, a tuple, a dictionary, a set, or a string

For example:

```
genre = ["Action", "Adventure", "Animation", "Biography", "Comedy", "Crime",  
"Drama", "Family", "Fantasy", "History", "Horror", "Music", "Musical", "Mystery",  
"Romance", "Sci-Fi", "Sport", "Thriller", "War", "Western"]
```

```
for value in genre:
```

```
    print(value)
```

```
    if value == "Comedy":
```

```
        break
```

Python: For Loop

a for loop iterates over a sequence

a list, a tuple, a dictionary, a set, or a string

For example:

```
for value in 'data science':  
    print(value)
```

Python: For Loop

`Range(num1:num2)`

returns numbers from num1 to num2 minus 1 (excluding num2)

For example:

```
base = input("Enter an integer base number")
```

```
exponent = input("Enter a positive integer exponent number")
```

```
result = 1
```

```
base = int(base)
```

```
exponent = int(exponent)
```

```
for i in range(0, exponent):
```

```
    result = result * base
```

```
print(result)
```


Python While Loop

Loops as long as a test condition is true

Python: For Loop

For example:

```
base = input("Enter an integer base number")
exponent = input("Enter a positive exponent number")
result = 1
```

```
base = int(base)
exponent = int(exponent)
```

```
for i in range(0, exponent):
    result = result * base
```

```
print(result)
```

For example:

```
base = input("Enter an integer base number")
exponent = input("Enter a positive exponent number")
result = 1
```

```
base = int(base)
exponent = int(exponent)
```

```
i = 0
while i < exponent:
    result = result * base
    i = i + 1
```

```
print(result)
```

Python: For Loop

For example:

```
base = input("Enter an integer base number")  
exponent = input ("Enter a positive integer exponent number")  
result = 1
```

```
base = int(base)  
exponent = int(exponent)
```

```
i = 0  
while i < exponent:  
    result = result * base  
    i = i + 1
```

```
print(result)
```

Data Science: Python Built-in Functions

<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	

Fruitful Function VS. Void Function (Procedure)

- Fruitful Function: returns a value to the caller
 - `Int()`
- Void Function (Procedure): perform tasks but not value returned to the caller
 - `Print()`

User-Defined Function (Fruitful Function)

- For example: we can turn our exponent code into a fruitful function:

```
def exponent(base, exp):  
    result = 1  
  
    for i in range(0, exp):  
        result = result * base  
  
    return result
```

Save this user-defined function as MyMathFunction.py in the same folder as the main program

```
import MyMathFunction  
  
base = input("Enter an integer base number")  
exp = input("Enter a positive integer exponent number")  
  
base = int(base)  
exp = int(exp)  
  
eresult = MyMathFunction.exponent(base, exp)  
  
print(base,"^",exp," = ",eresult)
```

This is the main program. Save this main program, in the same folder as MyMathFunction.py

User-Defined Function (Void Function or Procedure)

```
def Compute_Mean(numbers=[0,0,0]):  
    n = len(numbers)  
  
    total = 0  
    for value in numbers:  
        total = total + value  
  
    mean = total / n  
    return (mean)
```

```
from StatMean import Compute_Mean  
  
Compute_Mean([1,1,1])
```

This is the main program.

Save this user-defined function as StatMean.py in the same folder as the main program

Python Pandas DataFrame



DataFrame is a 2-dimensional labeled data structure with columns of potentially different types

Data Science: Python Pandas DataFrame

	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

Python Pandas DataFrame

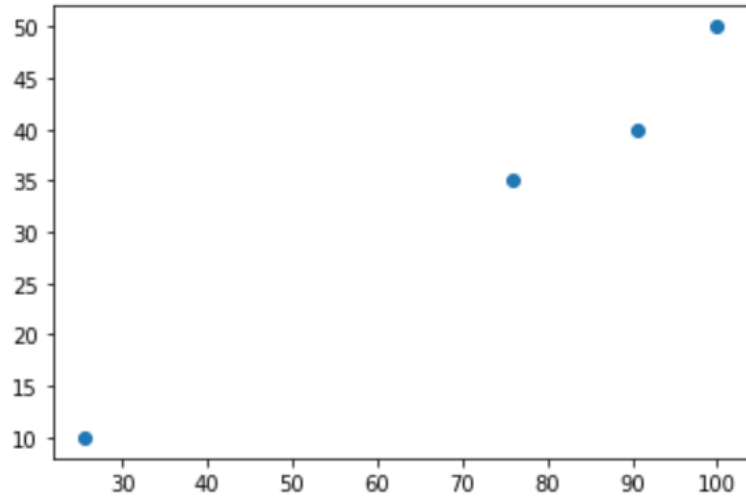
```
import pandas as pd
from matplotlib import pyplot as plt

grades = [90.5, 100.0, 75.8, 25.6]
studytime = [40, 50, 35, 10]
# Convert the List of Grades into Excel Spreadsheet Lookalike Column Format
# Use Pandas DataFrame to convert ONE list to start with (One Column)
df = pd.DataFrame(grades, columns = ["Grades"])
# Now we can add another List as second column to the dataframe created
df["Studytime"] = studytime
# Take a Look at the dataframe. Doesn't it look just like Excel?
print(df)
# Use Pandas DataFrame Correlation to perform Pearson R
print(df.corr())
plt.scatter(grades, studytime)
plt.show()
```

Data Science: Python Pandas DataFrame and Correlation

	Grades	Studytime
0	90.5	40
1	100.0	50
2	75.8	35
3	25.6	10

	Grades	Studytime
Grades	1.00000	0.99219
Studytime	0.99219	1.00000



Data Science: Python Pandas DataFrame and Correlation Demo

Python Pandas DataFrame

```
import pandas as pd
from matplotlib import pyplot as plt
%matplotlib inline

grades = [90.5, 100.0, 75.8, 25.6]
studytime = [40, 50, 35, 10]

# Convert the List of Grades into Excel Spreadsheet Lookalike Column Format
data = list(zip(grades, studytime))

df = pd.DataFrame(data, columns = ["Grades", "StudyTime"])

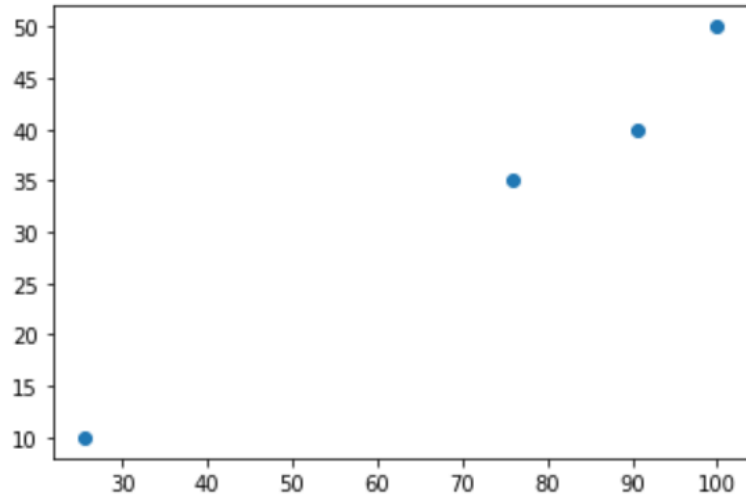
print (df)

# Use Pandas DataFrame Correlation
print (df.corr())
plt.scatter(studytime, grades)
plt.show()
```

Data Science: Python Pandas DataFrame and Correlation

	Grades	Studytime
0	90.5	40
1	100.0	50
2	75.8	35
3	25.6	10

	Grades	Studytime
Grades	1.00000	0.99219
Studytime	0.99219	1.00000



Python Pandas DataFrame

```
import pandas as pd
```

```
MyClass = {'students':['Bruce', 'Jane', 'Nancy',  
                        'Bill'],  
           'grades':[10, 9, 9, 8]}
```

```
df = pd.DataFrame(MyClass)
```

	students	grades
0	Bruce	10
1	Jane	9
2	Nancy	9
3	Bill	8

Python Pandas DataFrame

```
import pandas as pd
```

```
MyClass = {'students':['Bruce', 'Jane', 'Nancy',  
                        'Bill'],  
           'grades':[10, 9, 9, 8]}
```

```
df = pd.DataFrame(MyClass, index  
                  =["ID1","ID2","ID3","ID4"])
```

	students	grades
ID1	Bruce	10
ID2	Jane	9
ID3	Nancy	9
ID4	Bill	8

Python Pandas DataFrame

```
import pandas as pd
```

```
MyClass =
```

```
{'John':10,'Jake':9,'Jackie':8,'Jack':7,'Jane':6,'Jo':10,'Ja':9,'Jac':8,'Jacky':7,'Jan':6}
```

```
df = pd.DataFrame(MyClass, index=[1, 2, 3])
```

	John	Jake	Jackie	Jack	Jane	Jo	Ja	Jac	Jacky	Jan
1	10	9	8	7	6	10	9	8	7	6
2	10	9	8	7	6	10	9	8	7	6
3	10	9	8	7	6	10	9	8	7	6

Python Pandas DataFrame

```
MyInventory = {  
    "Item": ["coffee", "chocolate", "tea",  
            "water"],  
    "Promotion": [False, False, True, False],  
    "Price": [5.95, 5.95, 3.95, 2.95],  
    "Stock": [100, 250, 1000, 1200]  
}
```

```
ddf = pd.DataFrame(MyInventory)  
ddf
```

	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250
2	tea	True	3.95	1000
3	water	False	2.95	1200

Python Pandas DataFrame

```
Inv2 = {  
    "Item": ["coffee", "chocolate", "tea",  
            "water"],  
    "Promotion": ["no", "no", "yes", "yes"],  
    "Price": [5.95, 5.95, 3.95, 2.95],  
    "Stock": [100, 250, 1000, 1200]  
}
```

```
InvDF = pd.DataFrame(Inv2)  
InvDF
```

	Item	Promotion	Price	Stock
0	coffee	no	5.95	100
1	chocolate	no	5.95	250
2	tea	yes	3.95	1000
3	water	yes	2.95	1200

Python Pandas DataFrame

```
Inv2 = {  
    "Item": ["coffee", "chocolate", "tea",  
            "water"],  
    "Promotion": ["no", "no", "yes", "yes"],  
    "Price": [5.95, 5.95, 3.95, 2.95],  
    "Stock": [100, 250, 1000, 1200]  
}
```

```
InvDF = pd.DataFrame(Inv2)  
InvDF
```

```
InvDF = InvDF.replace({'Promotion': {'no':  
False, 'yes': True}})
```

	Item	Promotion	Price	Stock
0	coffee	no	5.95	100
1	chocolate	no	5.95	250
2	tea	yes	3.95	1000
3	water	yes	2.95	1200

	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250
2	tea	True	3.95	1000
3	water	True	2.95	1200

Python Pandas DataFrame

```
InvDF[InvDF["Promotion"] == False]
```

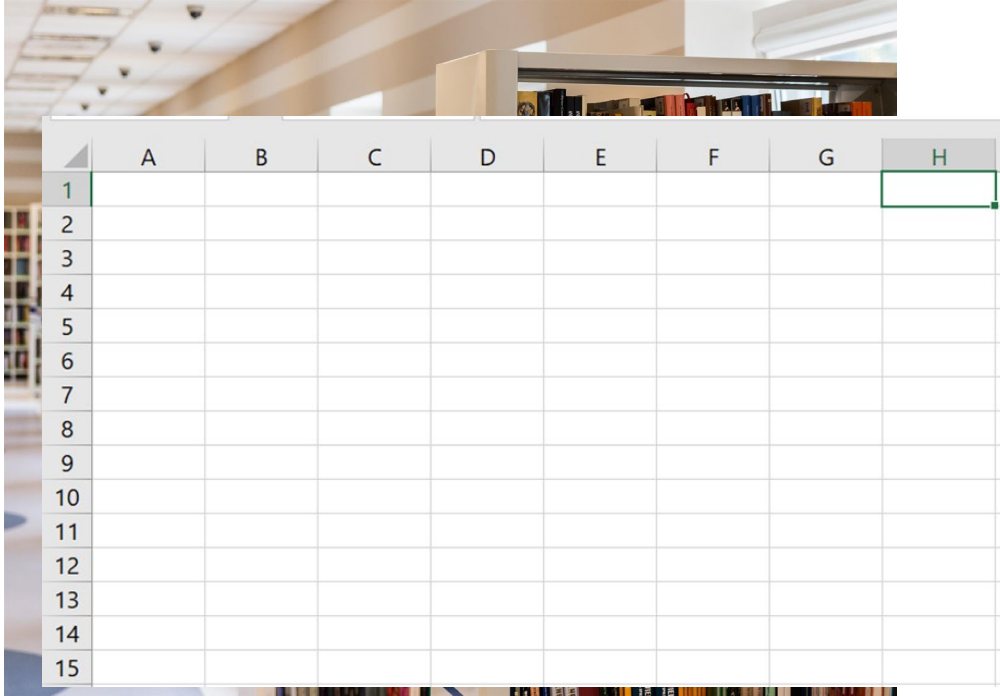
	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250

```
InvDF[InvDF["Price"] < 5]
```

	Item	Promotion	Price	Stock
2	tea	True	3.95	1000
3	water	True	2.95	1200

	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250
2	tea	True	3.95	1000
3	water	True	2.95	1200

Data Science: Python Pandas DataFrame from Excel

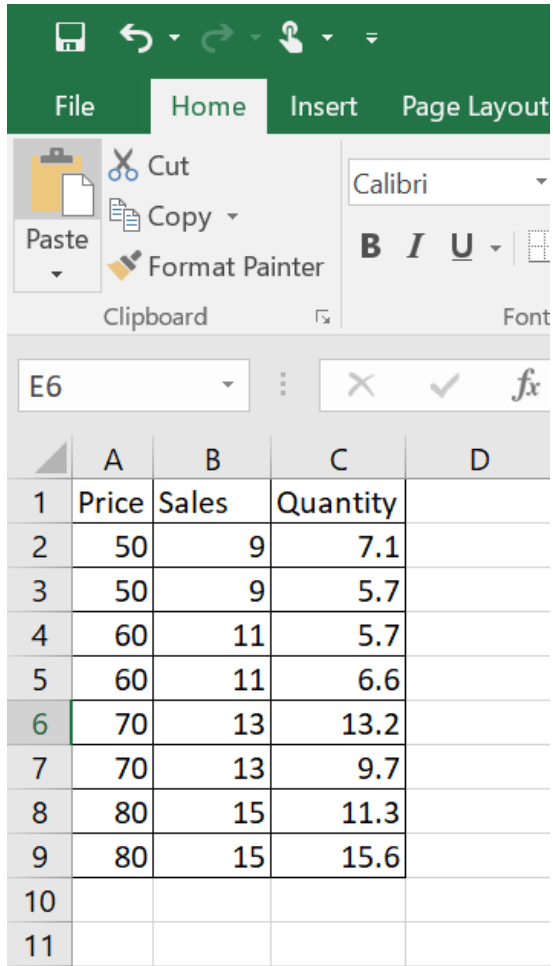


	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

Pandas **DataFrame** Review

DataFrame is a 2-dimensional labeled data structure with columns of potentially different types

Data Science: Python Pandas DataFrame from Excel



The screenshot shows the Microsoft Excel interface. The ribbon at the top includes 'File', 'Home', 'Insert', and 'Page Layout'. The 'Home' tab is active, showing the 'Clipboard' group with 'Cut', 'Copy', 'Paste', and 'Format Painter' options. The 'Font' group shows the font 'Calibri' and bold, italic, and underline buttons. The active cell is E6. The data is organized in a table with 4 columns: Price, Sales, Quantity, and an empty column D. The data is in rows 1-9, with row 6 highlighted.

	A	B	C	D
1	Price	Sales	Quantity	
2	50	9	7.1	
3	50	9	5.7	
4	60	11	5.7	
5	60	11	6.6	
6	70	13	13.2	
7	70	13	9.7	
8	80	15	11.3	
9	80	15	15.6	
10				
11				

Data Science: Python Pandas DataFrame from Excel

STEPS:

1. import the pandas library
2. Create a variable for the data frame to store all columns and values from the Excel worksheet
3. Use the API from pandas
`pandas.read_excel("filename.xlsx")`
to read the file "filename.xlsx" and assign all columns and values to the data frame variable

Data Science: Python Pandas DataFrame from Excel

```
In [19]: import pandas as pd

df = pd.read_excel ("QtyDemand.xlsx")

print(df)
```

	Price	Sales	Quantity
0	50	9	7.1
1	50	9	5.7
2	60	11	5.7
3	60	11	6.6
4	70	13	13.2
5	70	13	9.7
6	80	15	11.3
7	80	15	15.6

```
In [ ]:
```

Data Science: Python Pandas DataFrame from Excel

- You can access the specific column by referencing the index (column label) of that column
 - **df["Price"]** will return the values for the entire column "Price"
 - You can also use the form **df.Price**
- 0, 1, 2, 3, 4, 5, 6, 7 on the left most column is the index for the rows. You can access the value stored in column Price row 0 using **df.Price[index]**
 - **df.Price[0]** will return the value of the first element of column "Price" = 50
 - **df.Price[6]** will return the value of the seventh element of column "Price" = 80
 - **df.Sales[2] = 11**
 - **df.Quantity[4] = 13.2**

```
In [19]: import pandas as pd

df = pd.read_excel ("QtyDemand.xlsx")

print(df)
```

	Price	Sales	Quantity
0	50	9	7.1
1	50	9	5.7
2	60	11	5.7
3	60	11	6.6
4	70	13	13.2
5	70	13	9.7
6	80	15	11.3
7	80	15	15.6

```
In [ ]:
```

Data Science: Python Pandas DataFrame from Excel

```
In [22]: print(df.Price[0])
```

50

```
In [23]: print(df.Price[6])
```

80

```
In [24]: print(df.Sales[2])
```

11

```
In [25]: print(df.Quantity[4])
```

13.2

```
In [21]: print(df["Price"])
```

0 50

1 50

2 60

3 60

4 70

5 70

6 80

7 80

Name: Price, dtype: int64

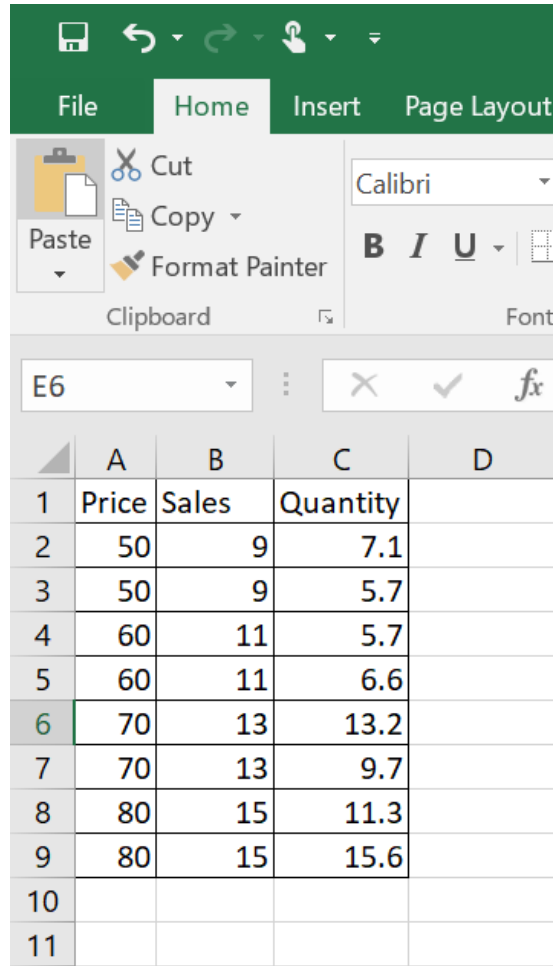
Data Science: Python Pandas DataFrame from Excel Columns



```
pandas.read_excel("filename.xlsx")
```

	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

Data Science: Python Pandas DataFrame from Excel



The screenshot shows the Microsoft Excel interface. The ribbon at the top includes 'File', 'Home', 'Insert', and 'Page Layout'. The 'Home' tab is active, showing the 'Clipboard' group with 'Cut', 'Copy', and 'Format Painter' options, and the 'Font' group with 'Calibri' font, bold (B), italic (I), and underline (U) options. The formula bar shows 'E6'. The data table is as follows:

	A	B	C	D
1	Price	Sales	Quantity	
2	50	9	7.1	
3	50	9	5.7	
4	60	11	5.7	
5	60	11	6.6	
6	70	13	13.2	
7	70	13	9.7	
8	80	15	11.3	
9	80	15	15.6	
10				
11				

What if we only want to read column
“Price” into the dataframe variable?

Data Science: Python Pandas DataFrame from Excel

STEPS:

1. import the pandas library
2. Create a variable for the data frame to store all columns and values from the Excel worksheet
3. Use the API from pandas
`pandas.read_excel("filename.xlsx")` to read the file "filename.xlsx" and assign all columns and values to the data frame variable.
 1. This time we will use an additional argument named `usecols`.
 2. `pandas.read_excel("filename.xlsx", usecols=[0])` to read first column only
 3. `pandas.read_excel("filename.xlsx", usecols=[0, 1])` to read first column and second column only

Data Science: Python Pandas DataFrame from Excel

```
import pandas as pd

df = pd.read_excel ("QtyDemand.xlsx", usecols = [0])

print(df)
```

	Price
0	50
1	50
2	60
3	60
4	70
5	70
6	80
7	80

```
import pandas as pd

df = pd.read_excel ("QtyDemand.xlsx", usecols = [0, 2])

print(df)
```

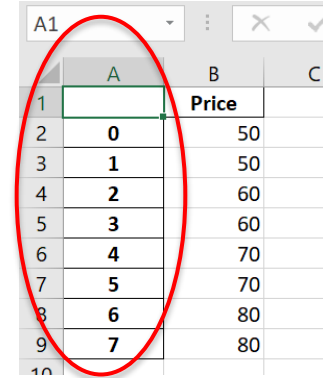
	Price	Quantity
0	50	7.1
1	50	5.7
2	60	5.7
3	60	6.6
4	70	13.2
5	70	9.7
6	80	11.3
7	80	15.6

Data Science: Python Pandas DataFrame Saving to Excel File

```
import pandas as pd  
df = pd.read_excel ("QtyDemand.xlsx", usecols = [0])  
print(df)
```

Price
0 50
1 50
2 60
3 60
4 70
5 70
6 80
7 80

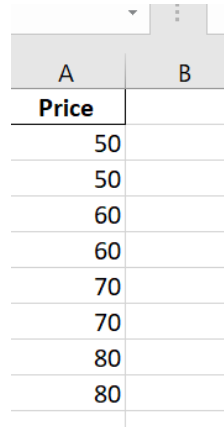
`df.to_excel ("test2.xlsx")`



An Excel spreadsheet with columns A, B, and C. Column A contains an index from 1 to 10. Column B contains the word 'Price' in the first row and values 50, 50, 60, 60, 70, 70, 80, 80 for rows 2 through 9. Column C is empty. A red circle highlights the index column (A).

	A	B	C
1		Price	
2	0	50	
3	1	50	
4	2	60	
5	3	60	
6	4	70	
7	5	70	
8	6	80	
9	7	80	
10			

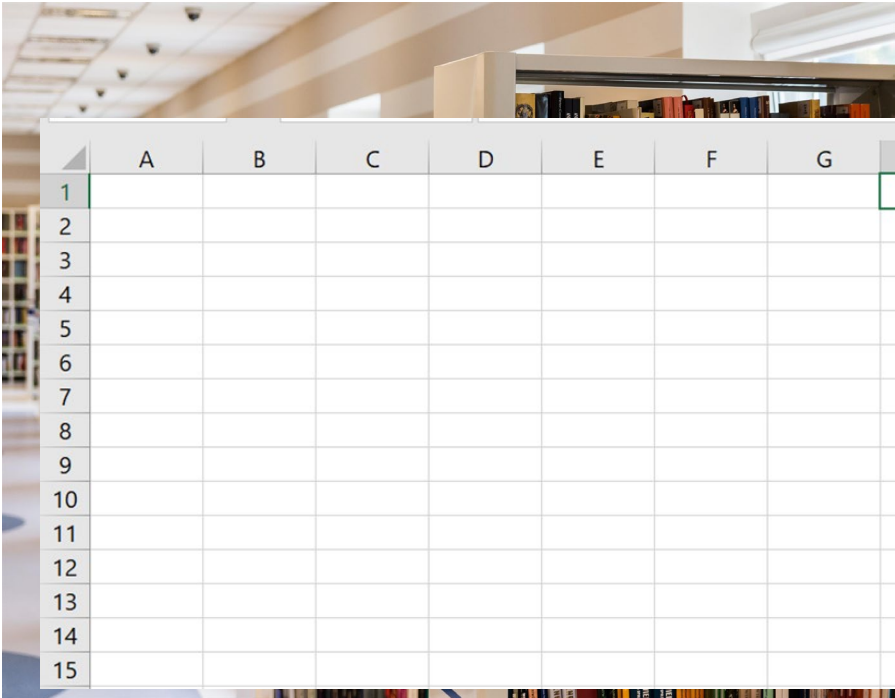
`df.to_excel ("test2.xlsx", index = False)`



An Excel spreadsheet with columns A and B. Column A contains the word 'Price' in the first row and values 50, 50, 60, 60, 70, 70, 80, 80 for rows 2 through 9. Column B is empty. The index column is omitted.

A	B
Price	
50	
50	
60	
60	
70	
70	
80	
80	

Data Science: Python Pandas DataFrame from csv



	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

Data Science: Python Pandas DataFrame from csv

```
Price,Sales,Quantity
```

```
50,9,7.1
```

```
50,9,5.7
```

```
60,11,5.7
```

```
60,11,6.6
```

```
70,13,13.2
```

```
70,13,9.7
```

```
80,15,11.3
```

```
80,15,15.6
```

Data Science: Python Pandas DataFrame from csv

STEPS:

1. import the pandas library
2. Create a variable for the data frame to store all columns and values from the csv file
3. Use the API from pandas `pandas.read_csv("filename.csv")` to read the file "filename.csv" and assign all data to the data frame variable

Data Science: Python Pandas DataFrame from csv

```
import pandas as pd

df = pd.read_csv ("QtyDemand.csv")

print(df)
```

	Price	Sales	Quantity
0	50	9	7.1
1	50	9	5.7
2	60	11	5.7
3	60	11	6.6
4	70	13	13.2
5	70	13	9.7
6	80	15	11.3
7	80	15	15.6

Data Science: Python Pandas DataFrame from csv

```
import pandas as pd

df = pd.read_csv ("QtyDemand.csv", usecols = [0])

print(df)
```

Specific Column

	Price
0	50
1	50
2	60
3	60
4	70
5	70
6	80
7	80

Data Science: Python Pandas DataFrame Saving to a csv file

```
import pandas as pd  
df = pd.read_csv ("QtyDemand.csv", usecols = [0])  
print(df)
```

Price
0 50
1 50
2 60
3 60
4 70
5 70
6 80
7 80



```
df.to_csv ("test3.csv")
```



A1	A	B	C
1		Price	
2	0	50	
3	1	50	
4	2	60	
5	3	60	
6	4	70	
7	5	70	
8	6	80	
9	7	80	
10			

```
df.to_csv ("test3.csv", index = False)
```



A	B
Price	
50	
50	
60	
60	
70	
70	
80	
80	